CIÊNCIA DA COMPUTAÇÃO / BANCO DE DADOS I

SQL em contexto

Laboratório da Universidade Aurora

Modelo relacional, dataset e 30 questões
com gabarito comentado ao final.

5 FÁCEIS	10 MÉDIAS	15 DIFÍCEIS
Fundamentos	Junções e conjuntos	Subqueries e quantificadores

9 relações · 308 registros · anos 2024 e 2025

RelaX para as consultas compatíveis.
PostgreSQL para a prática integral.

JOIN e OUTER JOIN / COALESCE / agregação
UNION, INTERSECT e EXCEPT / EXISTS, SOME e ALL
UNIQUE como restrição e como predicado do SQL padrão

Dados inteiramente fictícios. Edição de 30 de outubro de 2026.

01 / PREPARAÇÃO

Como usar o laboratório

O PDF contém os enunciados e as soluções. O kit anexo contém a base completa em dois formatos e os resultados de referência em CSV.

Prática integral no PostgreSQL

Execute `01_criar_e_popular_postgresql.sql` inteiro em um banco de testes. Ele cria o schema `treino_bd1`, as nove tabelas e os 308 registros em uma transação. Em cada nova sessão, execute:

```
SET search_path TO treino_bd1;
SELECT COUNT(*) FROM matricula; -- esperado: 174
```

O script não contém DROP e não cria um banco. Requer permissão para CREATE SCHEMA. Use o schema ainda sem estas tabelas: a carga não é idempotente. O comando completo para psql está no LEIA-ME.

Carregamento no RelaX

Abra dbis-uibk.github.io/relax. No **Group Editor** (editor de grupos), cole todo o arquivo `02_dataset_relax.txt`, carregue o grupo e abra a aba SQL. Execute uma consulta por vez. O formato de grupos descreve tabelas e dados; as restrições são aplicadas pelo PostgreSQL.

Compatibilidade indicada em cada questão

Selo	Onde executar	Questões
RX	RelaX e PostgreSQL	01-10, 13-15, 22 e 24
PG	PostgreSQL	11-12, 16-21, 23 e 25-30

O SQL do RelaX traduz consultas para álgebra relacional e tem limites documentados: EXISTS, IN, SOME/ANY e ALL em subqueries de predicado não são suportados. CTEs e subqueries no FROM são usadas nas questões RX compatíveis. Na Q10, o HAVING recebe o alias no RelaX; os dois gabaritos já incluem a sintaxe apropriada.

UNIQUE tem sentidos diferentes

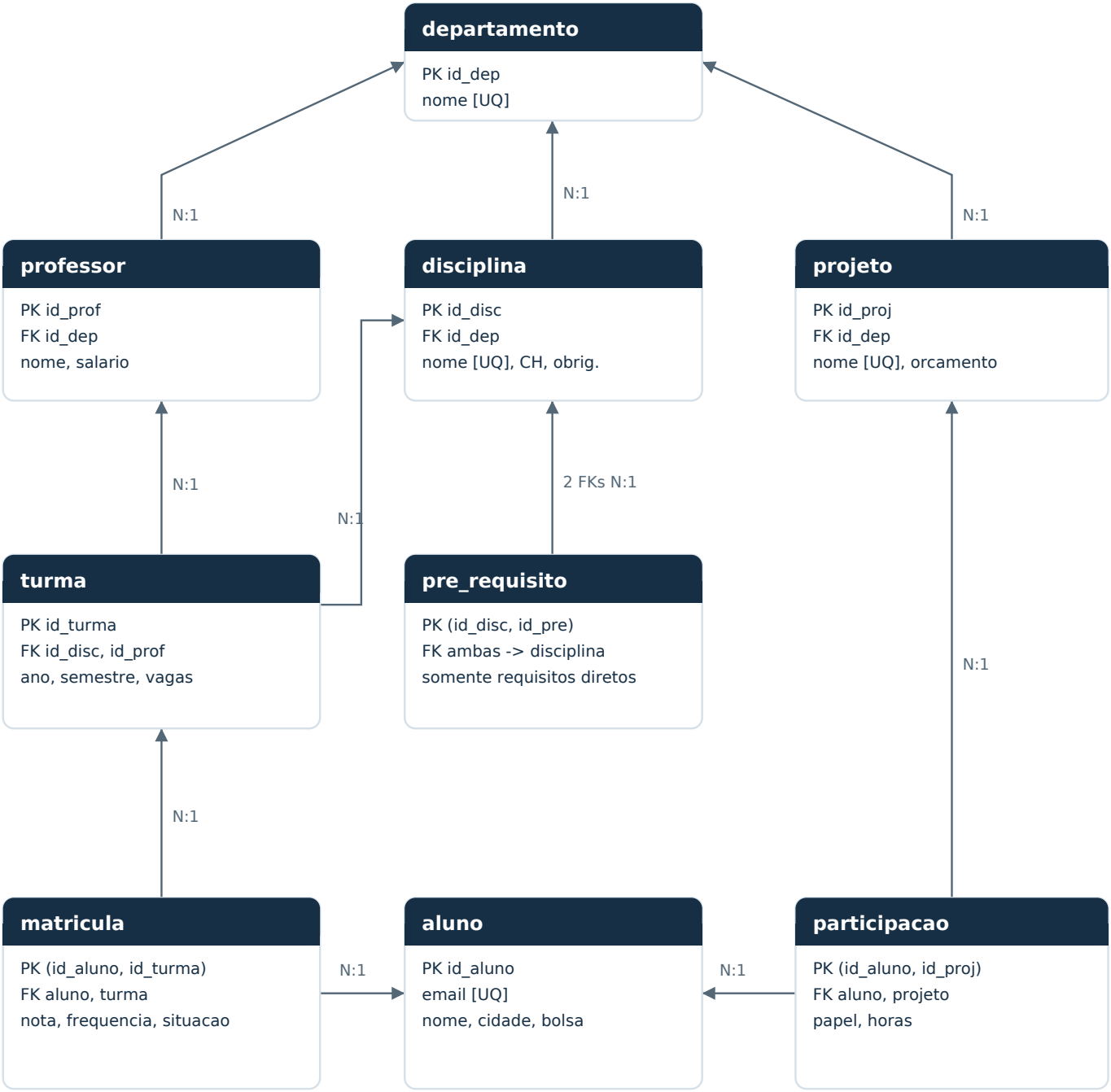
Restrição: protege valores únicos na tabela (ex.: aluno.email). **DISTINCT:** remove repetições de um resultado. **UNIQUE (subconsulta):** testa ausência de duplicatas no SQL padrão. Este último não é implementado no PostgreSQL nem no RelaX; a Q30 fornece o equivalente executável.

Tente cada questão antes de consultar o gabarito. Respeite colunas, períodos, empates, tratamento de NULL e ordenação solicitados.

02 / MODELO RELACIONAL

Mapa de relacionamentos

As setas saem da chave estrangeira e apontam para a chave referenciada. Cada ligação representa N:1; do lado pai, podem existir zero ou muitos filhos.



PK = chave primária; **FK** = chave estrangeira; **UQ** = restrição UNIQUE. As chaves de matrícula, participação e pré-requisito são compostas. CH = carga horária.

O vínculo do departamento de uma turma é obtido pela disciplina. O departamento do professor pode ser outro. As FKs e os tipos completos estão nas próximas páginas e no script SQL.

03 / DICIONÁRIO

Entidades principais

Todos os campos são NOT NULL, exceto os explicitamente indicados como anuláveis. As colunas que compõem uma PK são implicitamente NOT NULL.

departamento

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_dep	INTEGER	PK
nome	VARCHAR(60)	UNIQUE

professor

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_prof	INTEGER	PK
nome	VARCHAR(80)	Nome do docente
id_dep	INTEGER	FK -> departamento.id_dep
salario	NUMERIC(10,2)	Anulável; valor >= 0

aluno

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_aluno	INTEGER	PK
nome / cidade	VARCHAR(60)	Nome e cidade do aluno
bolsa	NUMERIC(10,2)	Anulável; valor >= 0
email	VARCHAR(100)	UNIQUE; domínio fictício .example

disciplina

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_disc	INTEGER	PK
nome	VARCHAR(80)	UNIQUE
id_dep	INTEGER	FK -> departamento.id_dep
carga_horaria	INTEGER	Valor > 0
obrigatoria	CHAR(1)	S ou N; núcleo do departamento

projeto

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_proj	INTEGER	PK
nome	VARCHAR(80)	UNIQUE
id_dep	INTEGER	FK -> departamento.id_dep
orcamento	NUMERIC(12,2)	Valor >= 0

03 / DICIONÁRIO

Vínculos e histórico

pre_requisito

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_disc	INTEGER	PK parcial; FK -> disciplina.id_disc
id_pre	INTEGER	PK parcial; FK -> disciplina.id_disc
PK / CHECK	Composta	(id_disc, id_pre); id_disc <> id_pre

turma

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_turma	INTEGER	PK
id_disc	INTEGER	FK -> disciplina.id_disc
id_prof	INTEGER	FK -> professor.id_prof
ano / semestre	INTEGER	Ano 2000-2100; semestre 1 ou 2
vagas	INTEGER	Valor > 0

matricula

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_aluno	INTEGER	PK parcial; FK -> aluno.id_aluno
id_turma	INTEGER	PK parcial; FK -> turma.id_turma
nota	NUMERIC(4,2)	Anulável; intervalo 0-10
frequencia	NUMERIC(5,2)	Anulável; percentual 0-100
situacao	VARCHAR(10)	APROVADO, REPROVADO, CURSANDO, TRANCADO

participacao

Campo	Tipo PostgreSQL	Chave / regra / nulabilidade
id_aluno	INTEGER	PK parcial; FK -> aluno.id_aluno
id_proj	INTEGER	PK parcial; FK -> projeto.id_proj
papel	VARCHAR(10)	BOLSISTA ou VOLUNTARIO
horas	NUMERIC(7,2)	Anulável; valor >= 0

As restrições não são regras acadêmicas completas. PK/FK garantem identidade e referência. A regra temporal de pré-requisitos é auditada na Q25; suas violações foram inseridas de propósito. A ausência de ciclos no grafo de pré-requisitos e a lotação das turmas não são garantidas por CHECK.

04 / DADOS E CONVENÇÕES

O que a base contém

Relação	Registros	Características de treino
departamento	4	Inclui Artes, sem docentes nem disciplinas.
professor	10	Há salário NULL, empate e docente sem turma.
aluno	30	Há alunos sem matrícula, sem projeto e sem ambos.
disciplina	12	Obrigatórias, optativas e disciplinas sem notas.
pre_requisito	10	Somente dependências diretas; uma disciplina pode ter duas.
turma	24	Ofertas em 2024/2025; turmas 23 e 24 estão vazias.
matricula	174	Aprovações, reprovações, novas tentativas e notas NULL.
projeto	8	Projetos 7 e 8 não têm participantes.
participacao	36	Há horas não informadas e vários projetos por aluno.

Período. Não use a data atual: os enunciados especificam 2024 ou 2025 e, quando necessário, o semestre.

Situação. APROVADO exige nota >= 7 e frequência >= 75. REPROVADO exige nota < 7 ou frequência < 75. Há reprovação por frequência mesmo com nota 8. Ambas as situações têm valores informados.

Nulos. CURSANDO e TRANCADO têm nota/frequência NULL. Bolsa NULL é valor não informado; bolsa 0 é ausência de bolsa. Horas NULL não apagam uma participação.

Repetições. Um aluno pode cursar uma disciplina em turmas distintas. Nunca use apenas o nome para identificar uma entidade.

Convenções. Bolsista nos filtros significa aluno.bolsa > 0. O papel no projeto é um atributo separado. O departamento responsável é o da disciplina ou do projeto, conforme pedido.

Saída. Ordene como solicitado. As médias exibidas no gabarito usam até quatro casas decimais; não arredonde antes das comparações. Os CSVs preservam a saída completa.

Ordem sugerida: Q01-Q05 para aquecimento; Q06-Q15 para composição; Q16-Q30 para correlação, conjuntos, quantificadores e nulos.

05 / ENUNCIADOS

Fáceis

Q01 | Bolsistas de Recife [RX]

Liste id_aluno, nome e bolsa dos alunos de Recife com bolsa maior que zero. Ordene por id_aluno.

SELECT, WHERE, ORDER BY

Q02 | Núcleo obrigatório [RX]

Liste id_disc, nome e carga_horaria das disciplinas obrigatórias do departamento 1 com pelo menos 60 horas. Ordene por carga_horaria decrescente e id_disc crescente.

Filtros e ordenação

Q03 | Catálogo de turmas [RX]

Para 2025.1, liste id_turma, nome da disciplina como disciplina e nome do professor como professor. Ordene por id_turma.

INNER JOIN

Q04 | Alunos por cidade [RX]

Conte todos os alunos de cada cidade. Retorne cidade e quantidade, em ordem alfabética de cidade.

COUNT, GROUP BY

Q05 | Notas ainda não lançadas [RX]

Liste id_aluno e id_turma das matrículas em situação CURSANDO cuja nota é NULL. Ordene por id_aluno e id_turma.

IS NULL

05 / ENUNCIADOS

Médias - parte 1

Q06 | Média e extremos das aprovações [RX]

Para cada disciplina com ao menos uma aprovação, calcule média, menor nota, maior nota e quantidade de matrículas APROVADO, considerando todos os anos. Retorne id_disc, nome, média, menor, maior e n_aprovacoes; ordene por id_disc. Não arredonde na consulta.

JOIN, AVG, MIN, MAX, COUNT

Q07 | Histórico de todos os alunos [RX]

Mostre todos os alunos, inclusive os sem matrículas, com id_aluno, nome e total_matriculas em qualquer ano. Ordene por id_aluno.

LEFT OUTER JOIN, COUNT

Q08 | Departamentos sem docentes [RX]

Encontre os departamentos sem nenhum professor. Retorne id_dep e nome, por id_dep. Resolva com LEFT JOIN e teste de NULL.

LEFT JOIN, antijunção

Q09 | Horas de pesquisa por aluno [RX]

Liste todos os alunos com id_aluno, nome e horas_projetos, somando as horas informadas em participação. Horas NULL e ausência de participação devem resultar em zero. Ordene por id_aluno.

COALESCE, subquery no FROM

Q10 | Mais de uma tentativa [RX]

Liste cada par aluno/disciplina com duas ou mais matrículas não trancadas em turmas diferentes, em qualquer ano. Retorne id_aluno, id_disc e tentativas, por esses dois IDs. Conte todas as situações, exceto TRANCADO.

GROUP BY, HAVING

05 / ENUNCIADOS

Médias - parte 2

Q11 | Acima da média global [PG]

Liste id_aluno, id_turma e nota das matrículas concluídas (APROVADO ou REPROVADO) cuja nota supera a média de todas as matrículas concluídas. Ordene pelos dois IDs.

Subquery escalar, AVG

Q12 | Aprovados em 2025 [PG]

Liste id_aluno e nome dos alunos com ao menos uma aprovação em turma de 2025, sem repetir alunos. Use EXISTS e ordene por id_aluno.

EXISTS correlacionado

Q13 | Computação ou pesquisa [RX]

Retorne somente id_aluno, sem repetições, dos alunos com alguma matrícula não trancada em disciplina do departamento 1, em qualquer ano, OU com alguma participação em projeto de qualquer departamento. Use UNION e ordene pelo ID.

UNION

Q14 | Duas disciplinas no mesmo ano [RX]

Retorne id_aluno dos alunos com matrícula não trancada tanto em Algoritmos (id_disc = 2) quanto em Banco de Dados I (id_disc = 3), em turmas de 2025. Use INTERSECT e ordene pelo ID.

INTERSECT

Q15 | Fora das turmas de 2025 [RX]

Retorne id_aluno dos alunos que não possuem nenhuma matrícula em turma de 2025, incluindo os que nunca se matricularam. Aqui até TRANCADO conta como matrícula. Use EXCEPT e ordene pelo ID.

EXCEPT

05 / ENUNCIADOS

Difíceis - parte 1

Q16 | Acima da média da própria disciplina [PG]

Liste id_aluno, id_turma, id_disc e nota das matrículas concluídas cuja nota supera a média das notas concluídas da mesma disciplina, considerando todas as turmas e todos os anos. Ordene pelos dois primeiros IDs.

Subquery correlacionada, AVG

Q17 | Todo o núcleo obrigatório [PG]

Liste id_aluno e nome de quem já foi aprovado em TODAS as disciplinas obrigatórias do departamento 1, em qualquer ano. Uma aprovação por disciplina basta. Use NOT EXISTS duplo e ordene por id_aluno.

Divisão relacional, NOT EXISTS duplo

Q18 | Históricos com conjuntos iguais [PG]

Encontre os alunos, exceto Alice (id_aluno = 1), cujo conjunto de disciplinas aprovadas em qualquer ano é exatamente igual ao dela. Ignore quantidade de tentativas e notas. Use diferença de conjuntos nos dois sentidos. Retorne id_aluno e nome, por ID.

EXCEPT em duas direções

05 / ENUNCIADOS

Difíceis - parte 2

Q19 | Aptos para Banco de Dados II [PG]

Considerando todo o histórico disponível, liste id_aluno e nome dos alunos aprovados em cada pré-requisito DIRETO de Banco de Dados II (id_disc = 5), mas ainda sem aprovação em BD II. Exija que a disciplina-alvo tenha ao menos um pré-requisito. Não teste disponibilidade de vagas. Ordene pelo ID.

EXISTS, NOT EXISTS, pré-requisitos

Q20 | Maior que algum colega [PG]

Liste id_prof, nome e salário dos professores cujo salário informado é maior que o de ALGUM OUTRO professor do mesmo departamento. Exclua salários NULL da comparação e use > SOME. Ordene por id_prof.

SOME / ANY

Q21 | Maiores salários com empates [PG]

Liste id_prof, nome, id_dep e salário dos professores com salário informado maior ou igual a TODOS os salários informados de seu departamento, incluindo o próprio na subquery. Preserve empates. Use >= ALL e ordene por id_dep e id_prof.

ALL, NULL, empates

05 / ENUNCIADOS

Difíceis - parte 3

Q22 | União de dois cadastros de atividade [RX]

Monte dois resumos: matrículas não trancadas de 2025 por aluno e participações em projetos por aluno. Combine-os com FULL OUTER JOIN. Retorne id_aluno, nome, n_matriculas, n_projetos e vinculo (AMBOS, SO_TURMA ou SO_PROJETO). Use zero nas contagens ausentes. Inclua apenas quem aparece em ao menos um resumo. Ordene por ID.

FULL OUTER JOIN, CTE, COALESCE

Q23 | Melhor média de turma por departamento [PG]

Entre as turmas de 2025 com pelo menos três matrículas concluídas, encontre a(s) de maior média de nota em cada departamento da DISCIPLINA. Inclua aprovados e reprovados, preserve empates e não arredonde no filtro. Retorne id_dep, id_turma e média, nessa ordenação.

CTE, HAVING, subquery correlacionada

Q24 | Interseção seguida de exclusão [RX]

Retorne id_aluno de quem teve alguma aprovação em 2024 E alguma matrícula não trancada em 2025, MAS nenhuma reprovação em 2025. Use INTERSECT e EXCEPT com agrupamento explícito; ordene pelo ID.

INTERSECT, EXCEPT, precedência

05 / ENUNCIADOS

Difíceis - parte 4

Q25 | Auditoria temporal de pré-requisitos [PG]

Para cada matrícula não trancada em turma de 2025, liste cada pré-requisito direto que o aluno NÃO havia aprovado em semestre estritamente anterior ao da turma auditada. Aprovação no mesmo semestre não basta. Retorne id_aluno, id_turma e id_pre como requisito_faltante, ordenados nessa sequência. A base contém violações intencionais.

NOT EXISTS correlacionado, ordem temporal

Q26 | Projetos que abrangem todos os bolsistas [PG]

Liste id_proj e nome dos projetos com participação de TODOS os alunos cuja bolsa informada é maior que zero. O papel no projeto é irrelevante e horas NULL não anulam a participação. Exija que exista ao menos um aluno com bolsa maior que zero; ordene pelo ID.

Quantificação universal, EXISTS

Q27 | Painel departamental sem multiplicação [PG]

Para TODOS os departamentos, retorne id_dep, nome, n_disciplinas, n_turmas_2025, n_alunos_2025 (alunos distintos com matrícula não trancada em disciplina do departamento, em 2025), media_final_2025 (notas APROVADO/REPROVADO de 2025) e horas_projetos (todos os projetos do departamento). Use zero para contagens e horas ausentes; preserve NULL na média sem notas. Não multiplique os agregados ao juntar as relações. Ordene pelo ID.

CTEs, COUNT DISTINCT, LEFT JOIN, AVG

05 / ENUNCIADOS

Difíceis - parte 5

Q28 | ALL com NULL e conjunto vazio [PG]

Para cada limite 6000, 9000 e 12000 e cada departamento 2 e 4, avalie limite > ALL (salários dos professores do departamento), SEM remover NULL. Retorne limite, id_dep e resultado como texto TRUE, FALSE ou UNKNOWN. Ordene por limite e id_dep. Explique a diferença entre um conjunto com NULL e um conjunto vazio.

ALL, lógica de três valores

Q29 | SOME e a perda de UNKNOWN no EXISTS [PG]

Repita os limites e departamentos da Q28 para limite > SOME (salários), preservando NULL. Retorne limite, id_dep, resultado_some (TRUE/FALSE/UNKNOWN) e resultado_exists, usando EXISTS para buscar algum salário menor que o limite. Ordene por limite e id_dep. Explique quando a reescrita preserva as linhas filtradas por WHERE, mas não o valor lógico completo.

SOME, EXISTS, NULL

Q30 | UNIQUE como predicado de subconsulta [PG]

Liste id_prof e nome dos professores que, em 2025, não ministraram a MESMA disciplina em mais de uma turma. Inclua quem não ministrou turma alguma. Escreva (a) a forma conceitual com UNIQUE (subconsulta) do SQL padrão e (b) uma equivalente executável em PostgreSQL, com NOT EXISTS, GROUP BY e HAVING. Ordene por id_prof. Não confunda UNIQUE com SELECT DISTINCT ou com restrição de tabela.

UNIQUE, NOT EXISTS, HAVING

06 / GABARITO

Soluções e conferência

Daqui em diante estão as consultas de referência. Há outras soluções corretas, desde que respeitem o enunciado e a multiplicidade das linhas.

Todas as 30 consultas e o script de carga foram executados em um motor PostgreSQL (PGlite). O dataset e as 15 consultas marcadas RX também foram executados no núcleo oficial do RelaX, com resultados equivalentes. A forma conceitual UNIQUE da Q30 não é executável nesses dois ambientes.

A conferência mostra a quantidade total de linhas de cada resultado. Resultados pequenos são exibidos por inteiro; os grandes têm amostra identificada. As saídas completas estão na pasta resultados/ do kit. Para conferir uma solução alternativa, compare também os valores e as repetições, não apenas a contagem.

Faixa	Questões e linhas esperadas
Q01-Q05	01: 6 / 02: 4 / 03: 9 / 04: 5 / 05: 40
Q06-Q10	06: 8 / 07: 30 / 08: 1 / 09: 30 / 10: 19
Q11-Q15	11: 74 / 12: 22 / 13: 29 / 14: 5 / 15: 4
Q16-Q20	16: 71 / 17: 9 / 18: 5 / 19: 9 / 20: 5
Q21-Q25	21: 4 / 22: 28 / 23: 3 / 24: 11 / 25: 19
Q26-Q30	26: 1 / 27: 4 / 28: 6 / 29: 6 / 30: 7

Fontes oficiais de sintaxe

RelaX: formato de grupos e referência SQL

RelaX: projeto e exemplos de datasets

PostgreSQL: EXISTS, SOME/ANY e ALL

PostgreSQL: recurso F291, UNIQUE predicate não implementado

PostgreSQL: COALESCE e CASE

HSQLDB: semântica de UNIQUE (subconsulta)

Referências consultadas em 30/10/2026. O recurso F291 é distinguido das restrições UNIQUE implementadas no PostgreSQL. Modelo, dados, questões e soluções deste laboratório são autorais.

Q01 | Bolsistas de Recife

RelaX + PostgreSQL · SELECT, WHERE, ORDER BY

```
SELECT id_aluno, nome, bolsa
     FROM aluno
     WHERE cidade = 'Recife' AND bolsa > 0
     ORDER BY id_aluno;
```

NULL não é maior que zero: a comparação é desconhecida e a linha não passa pelo WHERE.

Conferência: 6 linhas.

id_aluno	nome	bolsa
1	Alice	600
6	Felipe	600
11	Karen	600
17	Quelia	600
23	Wesley	600
29	Cintia	400

Q02 | Núcleo obrigatório

RelaX + PostgreSQL · Filtros e ordenação

```
SELECT id_disc, nome, carga_horaria
     FROM disciplina
     WHERE id_dep = 1 AND obrigatoria = 'S'
       AND carga_horaria >= 60
     ORDER BY carga_horaria DESC, id_disc;
```

O desempate pelo identificador torna a ordenação determinística.

Conferência: 4 linhas.

id_disc	nome	carga_horaria
1	Introducao a Programacao	60
2	Algoritmos	60
3	Banco de Dados I	60
4	Estruturas de Dados	60

Q03 | Catálogo de turmas

RelaX + PostgreSQL · INNER JOIN

```
SELECT t.id_turma, d.nome AS disciplina,
           p.nome AS professor
     FROM turma AS t
     JOIN disciplina AS d ON d.id_disc = t.id_disc
     JOIN professor AS p ON p.id_prof = t.id_prof
     WHERE t.ano = 2025 AND t.semestre = 1
     ORDER BY t.id_turma;
```

A disciplina e o professor são obtidos pelas respectivas chaves estrangeiras.

Conferência: 9 linhas.

id_turma	disciplina	professor
7	Introducao a Programacao	Ana Torres
8	Algoritmos	Bruno Melo
9	Banco de Dados I	Carla Luz
10	Banco de Dados I	Ana Torres
11	Estruturas de Dados	Bruno Melo
12	Calculo I	Diego Reis
13	Estatistica	Eva Nunes
14	Fisica	Gina Prado
15	Etica e Tecnologia	Fabio Leal

Q04 | Alunos por cidade

RelaX + PostgreSQL · COUNT, GROUP BY

```
SELECT cidade, COUNT(*) AS quantidade
     FROM aluno
     GROUP BY cidade
     ORDER BY cidade;
```

COUNT(*) conta linhas. Não se deve agrupar por nome do aluno.

Conferência: 5 linhas.

cidade	quantidade
Fortaleza	5
Joao Pessoa	5
Maceio	5
Natal	6
Recife	9

Q05 | Notas ainda não lançadas

RelaX + PostgreSQL · IS NULL

```
SELECT id_aluno, id_turma
     FROM matricula
     WHERE situacao = 'CURSANDO' AND nota IS NULL
     ORDER BY id_aluno, id_turma;
```

Use IS NULL, nunca = NULL. Uma linha por matrícula.

Conferência: 40 linhas.

id_aluno	id_turma
1	16
1	17
1	18
2	16
3	16

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q05.csv.

Q06 | Média e extremos das aprovações

RelaX + PostgreSQL · JOIN, AVG, MIN, MAX, COUNT

```
SELECT d.id_disc, d.nome, AVG(m.nota) AS media,
            MIN(m.nota) AS menor, MAX(m.nota) AS maior,
            COUNT(*) AS n_aprovacoes
     FROM disciplina AS d
     JOIN turma AS t ON t.id_disc = d.id_disc
     JOIN matricula AS m ON m.id_turma = t.id_turma
     WHERE m.situacao = 'APROVADO'
     GROUP BY d.id_disc, d.nome
     ORDER BY d.id_disc;
```

Cada aprovação participa da média. A consulta não inclui disciplinas sem aprovados.

Conferência: 8 linhas.

id_disc	nome	media	menor	maior	n_aprovacoes
1	Introducao a Programacao	7.96	7	9	25
2	Algoritmos	7.7647	7	9	17
3	Banco de Dados I	8	7	9	17
4	Estruturas de Dados	7.6667	7	8	9
9	Calculo I	7.8889	7	9	18
10	Estatistica	7.8333	7	9	12
11	Fisica	8	7	9	4
12	Etica e Tecnologia	8	7	9	7

Q07 | Histórico de todos os alunos

RelaX + PostgreSQL · LEFT OUTER JOIN, COUNT

```
SELECT a.id_aluno, a.nome,
           COUNT(m.id_turma) AS total_matriculas
     FROM aluno AS a
     LEFT OUTER JOIN matricula AS m
       ON m.id_aluno = a.id_aluno
     GROUP BY a.id_aluno, a.nome
     ORDER BY a.id_aluno;
```

COUNT(m.id_turma) devolve zero quando não há correspondência. COUNT(*) contaria a linha preservada pelo outer join.

Conferência: 30 linhas.

id_aluno	nome	total_matriculas
1	Alice	9
2	Beto	8
3	Celia	9
4	Davi	4
5	Elisa	7

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q07.csv.

Q08 | Departamentos sem docentes

RelaX + PostgreSQL · LEFT JOIN, antijunção

```
SELECT d.id_dep, d.nome
     FROM departamento AS d
     LEFT JOIN professor AS p ON p.id_dep = d.id_dep
     WHERE p.id_prof IS NULL
     ORDER BY d.id_dep;
```

Teste a chave primária do lado opcional, que é não nula em uma correspondência real.

Conferência: 1 linha.

id_dep	nome
4	Artes

Q09 | Horas de pesquisa por aluno

RelaX + PostgreSQL · COALESCE, subquery no FROM

```
SELECT a.id_aluno, a.nome,
            COALESCE(h.total, 0) AS horas_projetos
     FROM aluno AS a
     LEFT JOIN (
       SELECT id_aluno, SUM(horas) AS total
       FROM participacao
       GROUP BY id_aluno
     ) AS h ON h.id_aluno = a.id_aluno
     ORDER BY a.id_aluno;
```

SUM ignora horas NULL; se todas forem NULL, o resultado também é NULL. COALESCE cobre esse caso e a ausência de grupo.

Conferência: 30 linhas.

id_aluno	nome	horas_projetos
1	Alice	50
2	Beto	20
3	Celia	70
4	Davi	20
5	Elisa	0

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q09.csv.

Q10 | Mais de uma tentativa

RelaX + PostgreSQL · GROUP BY, HAVING

```
SELECT m.id_aluno, t.id_disc,
           COUNT(*) AS tentativas
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     WHERE m.situacao <> 'TRANCADO'
     GROUP BY m.id_aluno, t.id_disc
     HAVING COUNT(*) >= 2
     ORDER BY m.id_aluno, t.id_disc;
```

A chave composta impede duplicação na mesma turma, mas permite novas tentativas em outras turmas. No RelaX, use HAVING tentativas >= 2; o arquivo específico já traz essa adaptação.

Adaptação no RelaX: `HAVING tentativas >= 2`.

Conferência: 19 linhas.

id_aluno	id_disc	tentativas
4	1	2
5	2	2
5	9	2
7	1	2
8	3	2

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q10.csv.

Q11 | Acima da média global

PostgreSQL · Subquery escalar, AVG

```
SELECT m.id_aluno, m.id_turma, m.nota
     FROM matricula AS m
     WHERE m.situacao IN ('APROVADO','REPROVADO')
       AND m.nota > (
         SELECT AVG(m2.nota)
         FROM matricula AS m2
         WHERE m2.situacao IN ('APROVADO','REPROVADO')
       )
     ORDER BY m.id_aluno, m.id_turma;
```

A subquery escalar calcula uma única média global. A condição não usa apenas as aprovações.

Conferência: 74 linhas.

id_aluno	id_turma	nota
1	1	9
1	2	8
1	3	8
1	4	9
1	5	8

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q11.csv.

Q12 | Aprovados em 2025

PostgreSQL · EXISTS correlacionado

```
SELECT a.id_aluno, a.nome
     FROM aluno AS a
     WHERE EXISTS (
       SELECT 1
       FROM matricula AS m
       JOIN turma AS t ON t.id_turma = m.id_turma
       WHERE m.id_aluno = a.id_aluno
         AND m.situacao = 'APROVADO' AND t.ano = 2025
     )
     ORDER BY a.id_aluno;
```

EXISTS verifica existência; o número de correspondências não multiplica a linha externa.

Conferência: 22 linhas.

id_aluno	nome
2	Beto
4	Davi
5	Elisa
7	Giulia
8	Heitor

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q12.csv.

Q13 | Computação ou pesquisa

RelaX + PostgreSQL · UNION

```
SELECT m.id_aluno
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     JOIN disciplina AS d ON d.id_disc = t.id_disc
     WHERE d.id_dep = 1 AND m.situacao <> 'TRANCADO'
     UNION
     SELECT p.id_aluno FROM participacao AS p
     ORDER BY id_aluno;
```

UNION elimina repetições entre os dois conjuntos. UNION ALL teria outra multiplicidade.

Conferência: 29 linhas.

id_aluno: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29.

Q14 | Duas disciplinas no mesmo ano

RelaX + PostgreSQL · INTERSECT

```
SELECT m.id_aluno
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     WHERE t.id_disc = 2 AND t.ano = 2025
       AND m.situacao <> 'TRANCADO'
     INTERSECT
     SELECT m.id_aluno
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     WHERE t.id_disc = 3 AND t.ano = 2025
       AND m.situacao <> 'TRANCADO'
     ORDER BY id_aluno;
```

Os dois lados projetam a mesma coluna e filtram o mesmo ano.

Conferência: 5 linhas.

id_aluno: 19, 20, 21, 23, 26.

Q15 | Fora das turmas de 2025

RelaX + PostgreSQL · EXCEPT

```
SELECT id_aluno FROM aluno
     EXCEPT
     SELECT m.id_aluno
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     WHERE t.ano = 2025
     ORDER BY id_aluno;
```

O universo inicial precisa ser aluno, para incluir quem não aparece em matrícula.

Conferência: 4 linhas.

id_aluno: 16, 28, 29, 30.

Q16 | Acima da média da própria disciplina

PostgreSQL · Subquery correlacionada, AVG

```
SELECT m.id_aluno, m.id_turma, t.id_disc, m.nota
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     WHERE m.situacao IN ('APROVADO','REPROVADO')
       AND m.nota > (
         SELECT AVG(m2.nota)
         FROM matricula AS m2
         JOIN turma AS t2 ON t2.id_turma = m2.id_turma
         WHERE t2.id_disc = t.id_disc
           AND m2.situacao IN ('APROVADO','REPROVADO')
       )
     ORDER BY m.id_aluno, m.id_turma;
```

A correlação é pela disciplina, não pela turma. A própria nota faz parte da média de referência.

Conferência: 71 linhas.

id_aluno	id_turma	id_disc	nota
1	1	1	9
1	2	9	8
1	3	2	8
1	4	3	9
1	5	4	8

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q16.csv.

Q17 | Todo o núcleo obrigatório

PostgreSQL · Divisão relacional, NOT EXISTS duplo

```
SELECT a.id_aluno, a.nome
     FROM aluno AS a
     WHERE NOT EXISTS (
       SELECT 1 FROM disciplina AS d
       WHERE d.id_dep = 1 AND d.obrigatoria = 'S'
         AND NOT EXISTS (
           SELECT 1 FROM matricula AS m
           JOIN turma AS t ON t.id_turma = m.id_turma
           WHERE m.id_aluno = a.id_aluno
             AND t.id_disc = d.id_disc
             AND m.situacao = 'APROVADO'
         )
     )
     ORDER BY a.id_aluno;
```

Não pode existir disciplina obrigatória sem aprovação do aluno. Com conjunto exigido vazio, todos os alunos satisfariam a condição.

Conferência: 9 linhas.

id_aluno	nome
1	Alice
2	Beto
3	Celia
6	Felipe
8	Heitor
11	Karen
13	Marina
15	Olivia
17	Quelia

Q18 | Históricos com conjuntos iguais

PostgreSQL · EXCEPT em duas direções

```
WITH aprovadas AS (
       SELECT DISTINCT m.id_aluno, t.id_disc
       FROM matricula AS m
       JOIN turma AS t ON t.id_turma = m.id_turma
       WHERE m.situacao = 'APROVADO'
     )
     SELECT a.id_aluno, a.nome
     FROM aluno AS a
     WHERE a.id_aluno <> 1
       AND NOT EXISTS (
         SELECT x.id_disc FROM aprovadas AS x
         WHERE x.id_aluno = a.id_aluno
         EXCEPT
         SELECT y.id_disc FROM aprovadas AS y
         WHERE y.id_aluno = 1
       )
       AND NOT EXISTS (
         SELECT y.id_disc FROM aprovadas AS y
         WHERE y.id_aluno = 1
         EXCEPT
         SELECT x.id_disc FROM aprovadas AS x
         WHERE x.id_aluno = a.id_aluno
       )
     ORDER BY a.id_aluno;
```

Igualdade de conjuntos exige A menos B vazio E B menos A vazio. Mesma contagem não prova igualdade.

Conferência: 5 linhas.

id_aluno	nome
3	Celia
6	Felipe
8	Heitor
11	Karen
17	Quelia

Q19 | Aptos para Banco de Dados II

PostgreSQL · EXISTS, NOT EXISTS, pré-requisitos

```
SELECT a.id_aluno, a.nome
     FROM aluno AS a
     WHERE EXISTS (
       SELECT 1 FROM pre_requisito AS pr
       WHERE pr.id_disc = 5
     )
       AND NOT EXISTS (
         SELECT 1 FROM pre_requisito AS pr
         WHERE pr.id_disc = 5 AND NOT EXISTS (
           SELECT 1 FROM matricula AS m
           JOIN turma AS t ON t.id_turma = m.id_turma
           WHERE m.id_aluno = a.id_aluno
             AND t.id_disc = pr.id_pre
             AND m.situacao = 'APROVADO'
         )
       )
       AND NOT EXISTS (
         SELECT 1 FROM matricula AS m
         JOIN turma AS t ON t.id_turma = m.id_turma
         WHERE m.id_aluno = a.id_aluno
           AND t.id_disc = 5 AND m.situacao = 'APROVADO'
       )
     ORDER BY a.id_aluno;
```

Testam-se apenas requisitos diretos e aprovações, sem exigir que uma nova tentativa esteja ausente.

Conferência: 9 linhas.

id_aluno	nome
1	Alice
2	Beto
3	Celia
6	Felipe
8	Heitor
11	Karen
13	Marina
15	Olivia
17	Quelia

Q20 | Maior que algum colega

PostgreSQL · SOME / ANY

```
SELECT p.id_prof, p.nome, p.salario
     FROM professor AS p
     WHERE p.salario IS NOT NULL
       AND p.salario > SOME (
         SELECT p2.salario FROM professor AS p2
         WHERE p2.id_dep = p.id_dep
           AND p2.id_prof <> p.id_prof
           AND p2.salario IS NOT NULL
       )
     ORDER BY p.id_prof;
```

SOME é sinônimo de ANY. Se não houver outro salário informado, SOME sobre o conjunto vazio é FALSE.

Conferência: 5 linhas.

id_prof	nome	salario
1	Ana Torres	9500
2	Bruno Melo	11000
3	Carla Luz	11000
5	Eva Nunes	9500
8	Hugo Lima	9200

Q21 | Maiores salários com empates

PostgreSQL · ALL, NULL, empates

```
SELECT p.id_prof, p.nome, p.id_dep, p.salario
     FROM professor AS p
     WHERE p.salario IS NOT NULL
       AND p.salario >= ALL (
         SELECT p2.salario FROM professor AS p2
         WHERE p2.id_dep = p.id_dep
           AND p2.salario IS NOT NULL
       )
     ORDER BY p.id_dep, p.id_prof;
```

O filtro de NULL evita UNKNOWN. >= preserva os dois salários máximos empatados de Computação.

Conferência: 4 linhas.

id_prof	nome	id_dep	salario
2	Bruno Melo	1	11000
3	Carla Luz	1	11000
5	Eva Nunes	2	9500
8	Hugo Lima	3	9200

Q22 | União de dois cadastros de atividade

RelaX + PostgreSQL · FULL OUTER JOIN, CTE, COALESCE

```
WITH em_turma AS (SELECT m.id_aluno, COUNT(*) AS n
       FROM matricula AS m
       JOIN turma AS t ON t.id_turma = m.id_turma
       WHERE t.ano = 2025 AND m.situacao <> 'TRANCADO'
       GROUP BY m.id_aluno), em_projeto AS (SELECT id_aluno, COUNT(*) AS n
       FROM participacao GROUP BY id_aluno)
     SELECT a.id_aluno, a.nome,
       COALESCE(et.n, 0) AS n_matriculas,
       COALESCE(ep.n, 0) AS n_projetos,
       CASE WHEN (et.id_aluno IS NOT NULL
                  AND ep.id_aluno IS NOT NULL) THEN 'AMBOS'
            WHEN et.id_aluno IS NOT NULL THEN 'SO_TURMA'
            ELSE 'SO_PROJETO' END AS vinculo
     FROM em_turma AS et
     FULL OUTER JOIN em_projeto AS ep
       ON ep.id_aluno = et.id_aluno
     JOIN aluno AS a
       ON a.id_aluno = COALESCE(et.id_aluno, ep.id_aluno)
     ORDER BY a.id_aluno;
```

Agregue cada lado antes de juntar. Juntar as linhas brutas multiplicaria matrículas por participações.

Conferência: 28 linhas.

id_aluno	nome	n_matriculas	n_projetos	vinculo
1	Alice	3	3	AMBOS
2	Beto	4	2	AMBOS
3	Celia	3	3	AMBOS
27	Aline	2	0	SO_TURMA
28	Breno	0	1	SO_PROJETO
29	Cintia	0	2	SO_PROJETO

Amostra: primeiras 3 linhas e alunos 27, 28 e 29. Resultado completo: resultados/resultado_q22.csv.

Q23 | Melhor média de turma por departamento

PostgreSQL · CTE, HAVING, subquery correlacionada

```
WITH medias AS (
       SELECT d.id_dep, t.id_turma, AVG(m.nota) AS media
       FROM disciplina AS d
       JOIN turma AS t ON t.id_disc = d.id_disc
       JOIN matricula AS m ON m.id_turma = t.id_turma
       WHERE t.ano = 2025
         AND m.situacao IN ('APROVADO','REPROVADO')
       GROUP BY d.id_dep, t.id_turma
       HAVING COUNT(*) >= 3
     )
     SELECT x.id_dep, x.id_turma, x.media
     FROM medias AS x
     WHERE NOT EXISTS (
       SELECT 1 FROM medias AS y
       WHERE y.id_dep = x.id_dep AND y.media > x.media
     )
     ORDER BY x.id_dep, x.id_turma;
```

O departamento vem de disciplina, não do professor. A ausência de média maior preserva empates.

Conferência: 3 linhas.

id_dep	id_turma	media
1	15	8
2	13	7.5
3	14	7.6

Q24 | Interseção seguida de exclusão

RelaX + PostgreSQL · INTERSECT, EXCEPT, precedência

```
(
       SELECT m.id_aluno
       FROM matricula AS m
       JOIN turma AS t ON t.id_turma = m.id_turma
       WHERE t.ano = 2024 AND m.situacao = 'APROVADO'
       INTERSECT
       SELECT m.id_aluno
       FROM matricula AS m
       JOIN turma AS t ON t.id_turma = m.id_turma
       WHERE t.ano = 2025 AND m.situacao <> 'TRANCADO'
     )
     EXCEPT
     SELECT m.id_aluno
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     WHERE t.ano = 2025 AND m.situacao = 'REPROVADO'
     ORDER BY id_aluno;
```

Os parênteses registram a intenção: primeiro a interseção, depois a exclusão.

Conferência: 11 linhas.

id_aluno: 1, 2, 3, 5, 6, 8, 9, 11, 13, 14, 15.

Q25 | Auditoria temporal de pré-requisitos

PostgreSQL · NOT EXISTS correlacionado, ordem temporal

```
SELECT m.id_aluno, t.id_turma,
            pr.id_pre AS requisito_faltante
     FROM matricula AS m
     JOIN turma AS t ON t.id_turma = m.id_turma
     JOIN pre_requisito AS pr ON pr.id_disc = t.id_disc
     WHERE t.ano = 2025 AND m.situacao <> 'TRANCADO'
       AND NOT EXISTS (
         SELECT 1 FROM matricula AS m2
         JOIN turma AS t2 ON t2.id_turma = m2.id_turma
         WHERE m2.id_aluno = m.id_aluno
           AND t2.id_disc = pr.id_pre
           AND m2.situacao = 'APROVADO'
           AND (t2.ano < t.ano OR
                (t2.ano = t.ano AND t2.semestre < t.semestre))
       )
     ORDER BY m.id_aluno, t.id_turma, pr.id_pre;
```

Somente anos não resolvem a ordem temporal. Uma linha pode ser gerada para cada requisito faltante de uma matrícula.

Conferência: 19 linhas.

id_aluno	id_turma	requisito_faltante
4	10	1
7	10	1
12	10	1
14	18	4
18	10	1

Amostra: primeiras 5 linhas. Resultado completo: resultados/resultado_q25.csv.

Q26 | Projetos que abrangem todos os bolsistas

PostgreSQL · Quantificação universal, EXISTS

```
SELECT p.id_proj, p.nome
     FROM projeto AS p
     WHERE EXISTS (SELECT 1 FROM aluno WHERE bolsa > 0)
       AND NOT EXISTS (
         SELECT 1 FROM aluno AS a
         WHERE a.bolsa > 0 AND NOT EXISTS (
           SELECT 1 FROM participacao AS pa
           WHERE pa.id_proj = p.id_proj
             AND pa.id_aluno = a.id_aluno
         )
       )
     ORDER BY p.id_proj;
```

O EXISTS externo impede que todos os projetos sejam aceitos se não houver bolsistas. BOLSISTA no projeto e bolsa do aluno são atributos distintos.

Conferência: 1 linha.

id_proj	nome
1	Laboratorio Aberto

Q27 | Painel departamental sem multiplicação

PostgreSQL · CTEs, COUNT DISTINCT, LEFT JOIN, AVG

```
WITH nd AS (
       SELECT id_dep, COUNT(*) AS n_disciplinas
       FROM disciplina GROUP BY id_dep
     ), nt AS (
       SELECT d.id_dep, COUNT(*) AS n_turmas_2025
       FROM disciplina AS d
       JOIN turma AS t ON t.id_disc = d.id_disc
       WHERE t.ano = 2025 GROUP BY d.id_dep
     ), na AS (
       SELECT d.id_dep, COUNT(DISTINCT m.id_aluno) AS n
       FROM disciplina AS d
       JOIN turma AS t ON t.id_disc = d.id_disc
       JOIN matricula AS m ON m.id_turma = t.id_turma
       WHERE t.ano = 2025 AND m.situacao <> 'TRANCADO'
       GROUP BY d.id_dep
     ), nf AS (
       SELECT d.id_dep, AVG(m.nota) AS media
       FROM disciplina AS d
       JOIN turma AS t ON t.id_disc = d.id_disc
       JOIN matricula AS m ON m.id_turma = t.id_turma
       WHERE t.ano = 2025
         AND m.situacao IN ('APROVADO','REPROVADO')
       GROUP BY d.id_dep
     ), hp AS (
       SELECT p.id_dep, SUM(pa.horas) AS horas
       FROM projeto AS p
       JOIN participacao AS pa ON pa.id_proj = p.id_proj
       GROUP BY p.id_dep
     )
     SELECT d.id_dep, d.nome,
       COALESCE(nd.n_disciplinas, 0) AS n_disciplinas,
       COALESCE(nt.n_turmas_2025, 0) AS n_turmas_2025,
       COALESCE(na.n, 0) AS n_alunos_2025,
       nf.media AS media_final_2025,
       COALESCE(hp.horas, 0) AS horas_projetos
     FROM departamento AS d
     LEFT JOIN nd ON nd.id_dep = d.id_dep
     LEFT JOIN nt ON nt.id_dep = d.id_dep
     LEFT JOIN na ON na.id_dep = d.id_dep
     LEFT JOIN nf ON nf.id_dep = d.id_dep
     LEFT JOIN hp ON hp.id_dep = d.id_dep
     ORDER BY d.id_dep;
```

Cada CTE produz no máximo uma linha por departamento. COUNT DISTINCT evita contar o mesmo aluno em várias turmas. A média ausente não significa zero.

Conferência: 4 linhas.

id_dep	nome	n_disc.	n_turmas	n_alunos	média	horas
1	Computacao	9	13	26	7.5217	500
2	Matematica	2	3	14	7.4375	100
3	Engenharia	1	2	5	7.6	45
4	Artes	0	0	0	NULL	0

Q28 | ALL com NULL e conjunto vazio

PostgreSQL · ALL, lógica de três valores

```
WITH limites AS (
       SELECT 6000 AS valor UNION SELECT 9000
       UNION SELECT 12000
     ), deps AS (
       SELECT 2 AS id_dep UNION SELECT 4
     )
     SELECT l.valor AS limite, d.id_dep,
       CASE WHEN l.valor > ALL (
         SELECT p.salario FROM professor AS p
         WHERE p.id_dep = d.id_dep
       ) THEN 'TRUE'
       WHEN NOT (l.valor > ALL (
         SELECT p.salario FROM professor AS p
         WHERE p.id_dep = d.id_dep
       )) THEN 'FALSE'
       ELSE 'UNKNOWN' END AS resultado
     FROM limites AS l CROSS JOIN deps AS d
     ORDER BY l.valor, d.id_dep;
```

No departamento 2, salários são 7800, 9500 e NULL: 12000 gera UNKNOWN; os outros limites geram FALSE. No departamento 4, ALL vazio é TRUE.

Conferência: 6 linhas.

limite	id_dep	resultado
6000	2	FALSE
6000	4	TRUE
9000	2	FALSE
9000	4	TRUE
12000	2	UNKNOWN
12000	4	TRUE

Q29 | SOME e a perda de UNKNOWN no EXISTS

PostgreSQL · SOME, EXISTS, NULL

```
WITH limites AS (
       SELECT 6000 AS valor UNION SELECT 9000
       UNION SELECT 12000
     ), deps AS (
       SELECT 2 AS id_dep UNION SELECT 4
     )
     SELECT l.valor AS limite, d.id_dep,
       CASE WHEN l.valor > SOME (
         SELECT p.salario FROM professor AS p
         WHERE p.id_dep = d.id_dep
       ) THEN 'TRUE'
       WHEN NOT (l.valor > SOME (
         SELECT p.salario FROM professor AS p
         WHERE p.id_dep = d.id_dep
       )) THEN 'FALSE'
       ELSE 'UNKNOWN' END AS resultado_some,
       CASE WHEN EXISTS (
         SELECT 1 FROM professor AS p
         WHERE p.id_dep = d.id_dep AND l.valor > p.salario
       ) THEN 'TRUE' ELSE 'FALSE' END AS resultado_exists
     FROM limites AS l CROSS JOIN deps AS d
     ORDER BY l.valor, d.id_dep;
```

6000 no departamento 2 produz SOME = UNKNOWN e EXISTS = FALSE. Um WHERE positivo rejeita ambos; sob NOT, a diferença importa. SOME vazio é FALSE.

Conferência: 6 linhas.

limite	id_dep	resultado_some	resultado_exists
6000	2	UNKNOWN	FALSE
6000	4	FALSE	FALSE
9000	2	TRUE	TRUE
9000	4	FALSE	FALSE
12000	2	TRUE	TRUE
12000	4	FALSE	FALSE

Q30 | UNIQUE como predicado de subconsulta

PostgreSQL · UNIQUE, NOT EXISTS, HAVING

(a) SQL padrão, apenas para estudo do predicado:

```
SELECT p.id_prof, p.nome
FROM professor AS p
WHERE UNIQUE (
  SELECT t.id_disc FROM turma AS t
  WHERE t.id_prof = p.id_prof AND t.ano = 2025
)
ORDER BY p.id_prof;
```

(b) Equivalente executável em PostgreSQL:

```
SELECT p.id_prof, p.nome
     FROM professor AS p
     WHERE NOT EXISTS (
       SELECT t.id_disc FROM turma AS t
       WHERE t.id_prof = p.id_prof AND t.ano = 2025
       GROUP BY t.id_disc
       HAVING COUNT(*) > 1
     )
     ORDER BY p.id_prof;
```

A subconsulta conceitual projeta id_disc, que é NOT NULL. UNIQUE aceita resultado vazio; DISTINCT eliminaria justamente as repetições que precisam ser detectadas. PostgreSQL não implementa esse predicado.

Conferência: 7 linhas.

id_prof	nome
3	Carla Luz
4	Diego Reis
6	Fabio Leal
7	Gina Prado
8	Hugo Lima
9	Iris Costa
10	Joao Paiva